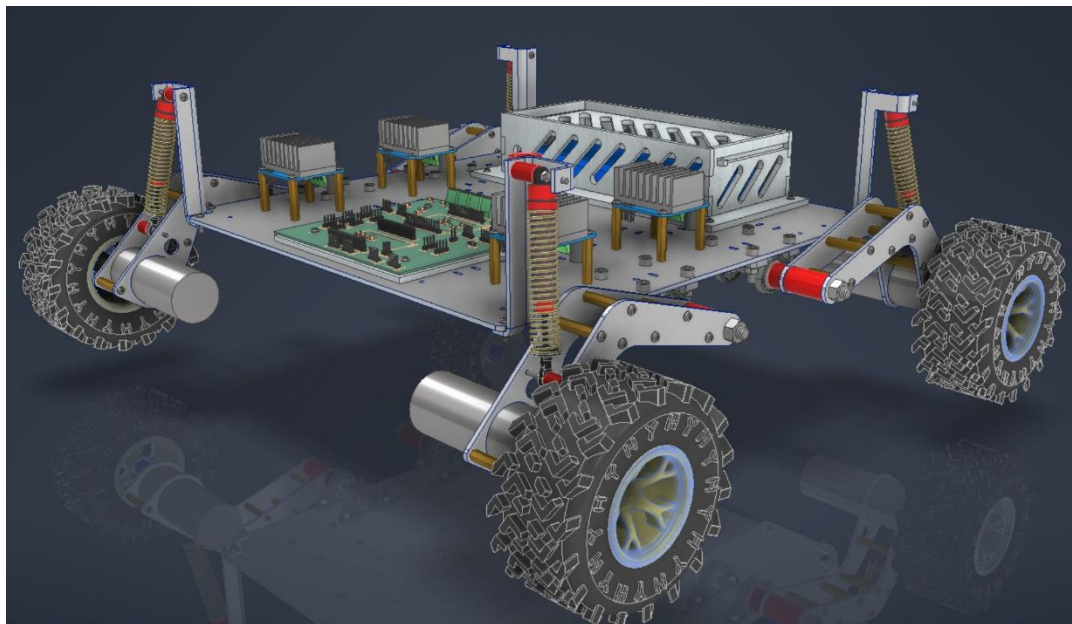


FINITE ELEMENT ANALYSIS OF A PLATE USED IN SWARM ROBOT SURVEYING



Name	ID	Contribution
Omar Mohamed Sayed	2200198	30%
Moaz Ahmed Foad	2200790	25%
Taha gamal	2100566	25%
Youssef Khairat	2200845	20%

Table of Contents

I.	Introduction	5
	The Evolution of Land Surveying	5
	The Proposed Swarm Solution: A Multi-Agent Paradigm	6
	Mechanical Innovation for Data Integrity	7
II.	Problem Identification (Theory)	7
	1. Problem Definition.....	7
	2. Load Identification.....	8
	3. Thin-Plate Verification	8
	4. Kirchhoff Plate Assumptions	9
	5. Element Selection and Degrees of Freedom	10
	6. Displacement Field (Polynomial Selection via Pascal's Triangle).....	11
	7. Curvatures	12
	8. Justification: Plate Bending Element vs. Beam / Shell	13
III.	PrePro.....	14
	Boundary Condition	14
	Fixation	15
	Loading	15
	Drawing on Free Cad	15
	Meshing on Gmsh	16
	PostProcessing on PrePro	17
IV.	MATLAB.....	18
	Numerical method	18
	MATLAB Results.....	19
	Appendix A — MATLAB Implementation of the Plate Bending Element.....	20
	A.1 Overview of the Solution Procedure.....	20

A.2 Explanation of the Main Code Blocks.....	20
Material properties and flexural rigidity.....	20
Mesh import and degrees of freedom.....	20
Loads and boundary conditions.....	21
Gauss quadrature	21
Element stiffness matrix assembly	21
Solving the system and extracting deflections	21
Stress recovery and safety factor	21
Post-processing.....	21
The shape-function core: P_matrix and Q_matrix.....	22
A.3 Mapping Between Code and Theory	22
A.4 Complete MATLAB Listing	23

Table of Figures

Figure 1 Traditional method	5
Figure 2 Syrveing Swarm	6
Figure 3 Plate	7
Figure 4Kirchoff Plate Theory	9
Figure 5 thickness perserved	9
Figure 6 Plate Element DOF	10
Figure 7 Pascal Triangle	11
Figure 8 Strain Curvature	12
Figure 9 Shell vs Plate Bending Element	13
Figure 10 Boundary Conditions	14
Figure 11 Free CAD	15
Figure 12 Gmsh	16
Figure 13 prepro results	17
Figure 14 Gaussian	18
Figure 15parameters	18
Figure 16 Deflection Result MATLAB	19
Figure 17VON MISES Results MATLAB	19

I. Introduction

The Evolution of Land Surveying

Land surveying is among the oldest engineering professions, with roots stretching back to the Bronze Age. Historically, the discipline has relied on human-centric labor to establish the 3D positions of points and calculate the precise distances and angles between them. While tools like the theodolite significantly improved accuracy, the core process remained labor-intensive and time-consuming.

In the 21st century, the industry is undergoing a momentous shift toward automation. Robotic stations and drones have begun to replace manual foot surveys, reducing task duration from weeks to days and minimizing human error through automated mathematical processing. However, as the industry becomes more tech-oriented, the focus is shifting from field labor to high-level data management and the maintenance of robotic fleets.



Figure 1 Traditional method

The Proposed Swarm Solution: A Multi-Agent Paradigm

To address the limitations of current single-agent technology, this project introduces a novel approach: The Surveying Swarm. To our knowledge, while research into swarm robotics exists for planetary exploration, there is currently **no** commercially available swarm of ground-based robots specifically for land surveying.

This project implements a decentralized collective of three cost-effective, wheeled differential-drive robots. By shifting from a single "super-robot" to a "swarm," we achieve:

- **Efficiency through Parallelism:** Three robots can survey different sectors of a terrain simultaneously, reducing mission time by up to 66%.
- **Systemic Redundancy:** Unlike the single-agent platforms mentioned above, our system is resilient to failure; if one robot is compromised, the remaining two can recalibrate to finish the survey.
- **Collaborative Mapping:** Through shared localization and map-merging techniques, the swarm navigates unknown environments and avoids obstacles collectively.



Figure 2 Syrveing Swarm

Mechanical Innovation for Data Integrity

A primary challenge in outdoor surveying is "sensor noise" caused by uneven terrain. To ensure our data is superior to basic autonomous rovers, our robots feature a custom-engineered suspension system. This provides passive vibration attenuation, which is critical for sensor fusion. By stabilizing the chassis, we ensure cleaner sensory data, allowing for more precise 3D point calculations even on rough terrain.

*Upon this we have selected to study the plate and make FEA on the plate to get the thickness through the maximum **accepted deflection to the plate (1mm)**.*

II. Problem Identification (Theory)

1. Problem Definition

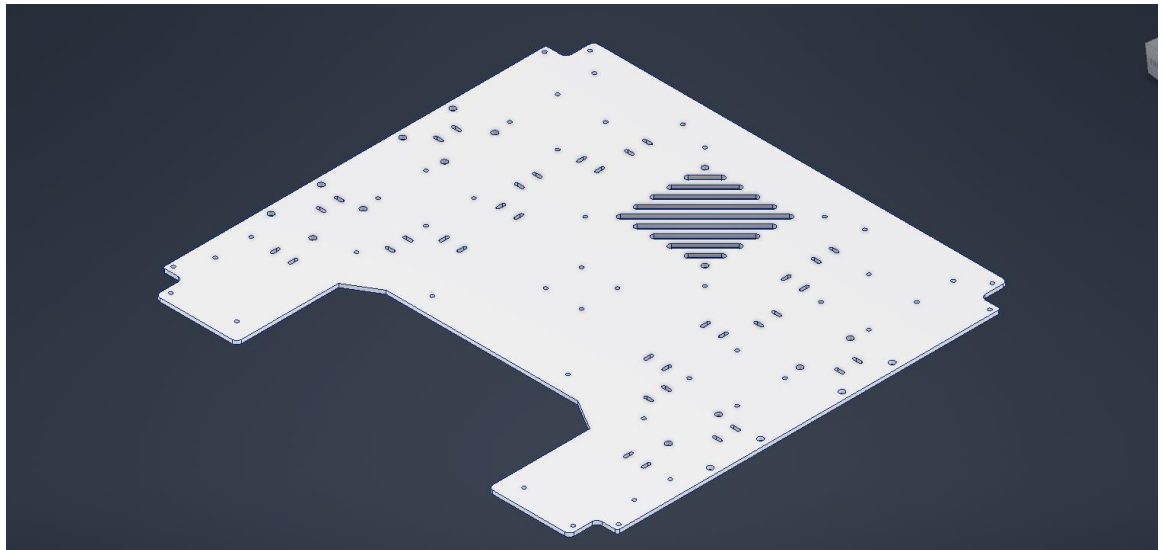


Figure 3 Plate

The component under study is a plate taken from our mobile robot. The first objective of the analysis is to determine the required thickness of this plate. Before any modelling can begin, we need to establish two things: ***the type of loading acting on the plate***, and whether the plate can be treated as ***thin***. These two points determine which plate theory and which finite element we are allowed to use.

2. Load Identification

Examining the loading condition, we found that the plate carries a ***lateral load*** only — that is, a load acting perpendicular to the plane of the plate. There is ***no significant in-plane loading***, so no membrane tension or compression acts within the plane of the plate. This is the first key observation: the loading is ***purely transverse (bending-type)***, with no in-plane component.

3. Thin-Plate Verification

To decide whether classical thin-plate theory applies, we compared the plate thickness against its in-plane dimensions. We found that even if the thickness is multiplied by ten, it is still smaller than the plate's length and width. The thickness is therefore very small relative to the in-plane dimensions, which allows us to classify the component as a ***thin plate***.

So at this stage we have established:

- The load is lateral (transverse bending load).
- The geometry qualifies as a thin plate.

On this basis, we select ***Classical (Kirchhoff) Thin-Plate Theory*** as the governing theory and a ***plate bending element*** for the analysis.

4. Kirchhoff Plate Assumptions

Kirchhoff-Love plate theory

1. Normal fibers remain *straight* and *perpendicular* to the mid-surface
 - Shear deformations are negligible for *thin* plates
2. Inextensional bending deformations caused by transverse loading only
3. Small deflections (and rotations)

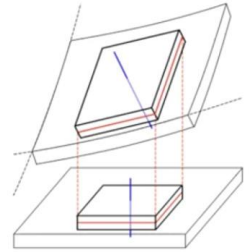


Figure 4 Kirchhoff Plate Theory

Classical thin-plate theory is built on the Kirchhoff assumptions. Its core requirement is simply that the plate be thin — which we have already confirmed. The theory rests on the following assumptions:

- **Normals remain normal.** A straight fibre that is perpendicular (normal) to the mid-surface before bending remains straight and perpendicular to the deformed mid-surface after bending. (This also implies the transverse shear strains are neglected.)
- **Thickness change is neglected.** The plate thickness does not change during deformation, which means the transverse normal strain is zero: $\epsilon_z = 0$. Physically, even when the plate bends, its through-thickness dimension stays the same — the same set of fibres simply rotates/translates without stretching in the z -direction. Since $\epsilon_z = 0$ and we are designing on this basis, the transverse normal strain has no influence on the result.

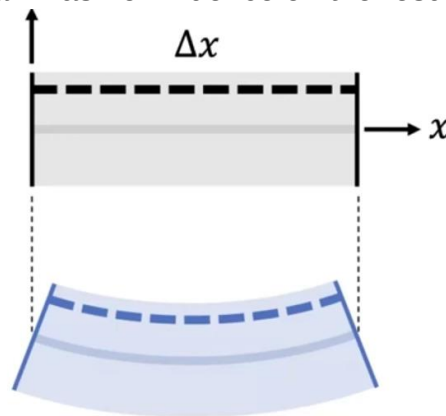


Figure 5 thickness preserved

- **Transverse normal stress is negligible** (consistent with assumption 2 for a thin plate).
- **No in-plane (membrane) strains.** Because the applied load is purely lateral, it produces no in-plane stretching of the mid-surface. The in-plane mid-surface displacements $\mathbf{u}$ and $\mathbf{v}$ are therefore zero, and there are no membrane strains in the x- or y-directions.

5. Element Selection and Degrees of Freedom

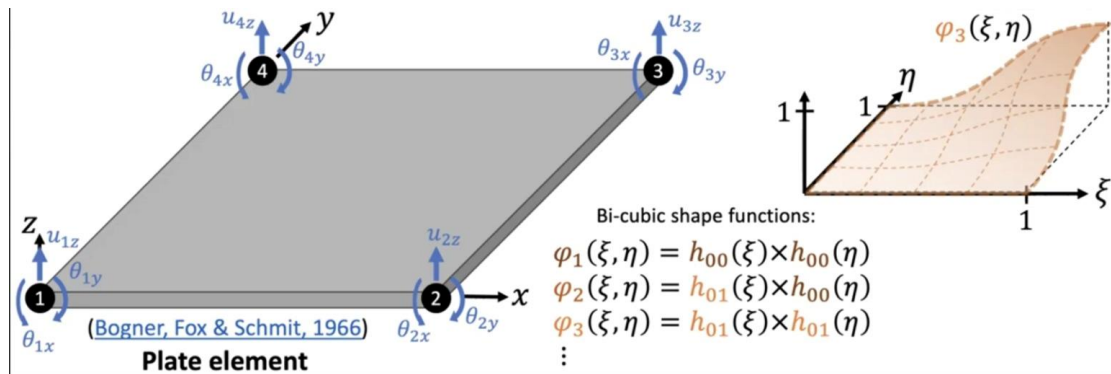


Figure 6 Plate Element DOF

Based on the above, we use a **quadrilateral plate bending element** with **4 nodes**. Each node has **3 degrees of freedom**, giving a total of **12 degrees of freedom** per element. The three DOF at each node are:

- w — the transverse (out-of-plane) displacement in the z-direction.
- θ_x — bending rotation about the x-axis.
- θ_y — bending rotation about the y-axis.

These are exactly the quantities needed to describe lateral bending, and nothing more — which is consistent with having ignored in-plane deformation.

6. Displacement Field (Polynomial Selection via Pascal's Triangle)

Since the element has **12 degrees of freedom**, the assumed displacement function for **w** must contain **12 unknown constants**. The displacement field must be rich enough to properly represent plate bending — and in particular the **curvature**, which is the central quantity in plate bending.

To choose a complete and balanced set of polynomial terms, we use **Pascal's triangle**. Working down the triangle:

- 1 constant term,
- 2 linear terms,
- 3 quadratic terms,
- 4 cubic terms,

gives **10 terms**. We still need **2** more, which are taken from the **quartic (fourth-order) row**. Pascal's triangle offers 5 quartic terms, but we deliberately select only **x^3y** and **xy^3** .

This specific choice is made to **inter-element displacement continuity (compatibility)**. On an edge like $y = 0$, the terms **x^3y** and **xy^3** vanish, so they add no variation along that edge and stay compatible with the neighboring element. Including **x^4 (or y^4)** leaves a quartic variation along the edge that neighbors can't reproduce → displacement discontinuity, so they're rejected. **x^2y^2** is also rejected because its second derivatives make the curvature quadratic rather than controlled/linear.

The resulting 12-term polynomial is:

$$w = a_1 + a_2x + a_3y + a_4x^2 + a_5xy + a_6y^2 + a_7x^3 + a_8x^2y + a_9xy^2 + a_{10}y^3 + a_{11}x^3y + a_{12}xy^3$$

For FEM, Pascal's triangle is simply a way to list all possible polynomial terms.

Order 0:	1
Order 1:	x y
Order 2:	x ² xy y ²
Order 3:	x ³ x ² y xy ² y ³
Order 4:	x ⁴ x ³ y x ² y ² xy ³ y ⁴

Figure 7 Pascal Triangle

7. Curvatures

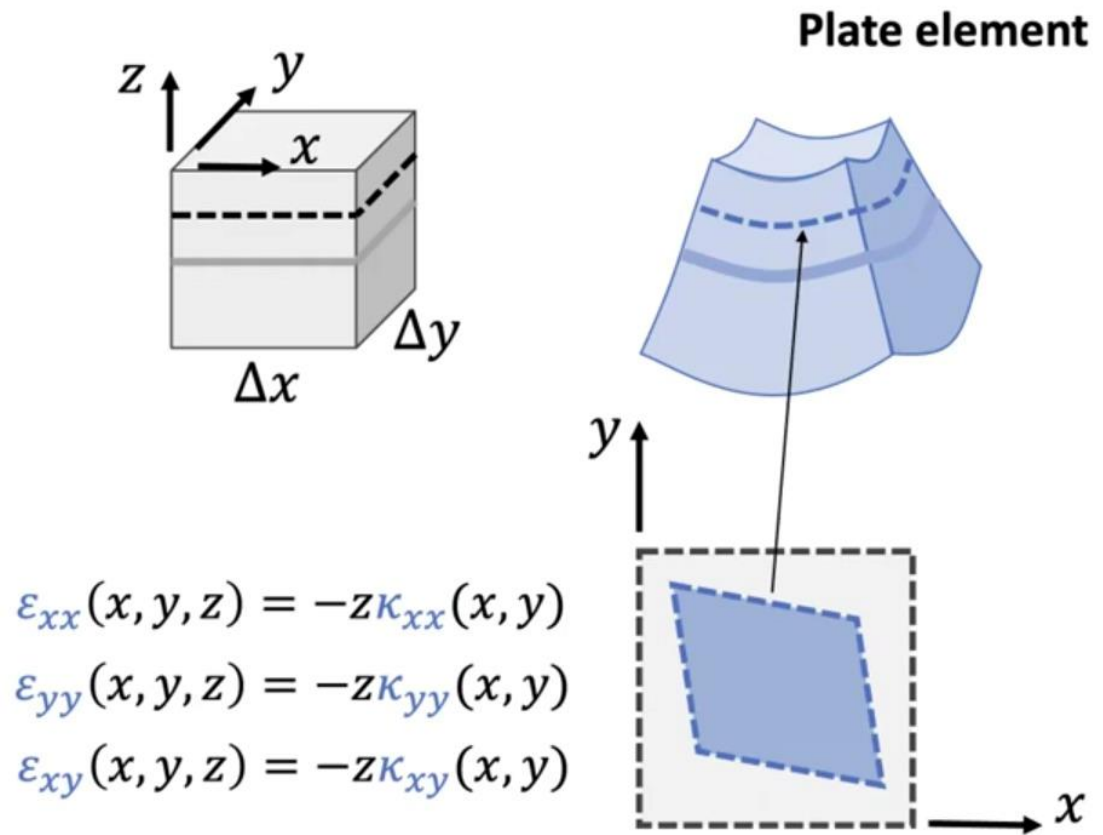


Figure 8 Strain Curvature

The reason the displacement polynomial matters so much is that the **curvatures** are obtained from the second derivatives of w . As introduced at the start of the theory, the plate curvatures are:

- $\kappa_x = \partial^2 w / \partial x^2$ (curvature in the x-direction)
- $\kappa_y = \partial^2 w / \partial y^2$ (curvature in the y-direction)
- $\kappa_{xy} = 2 \cdot \partial^2 w / \partial x \partial y$ (twisting curvature)

The term that produces a given curvature is named accordingly — for example, x^2 is what describes the curvature in x , which is why it is associated with the x-curvature. The chosen polynomial ensures every required curvature term is correctly represented.

8. Justification: Plate Bending Element vs. Beam / Shell

Finally, the choice of element type is justified as follows. A **beam element** is one-dimensional and cannot capture two-dimensional plate bending. A **shell element** is more general — it carries both bending and in-plane (membrane) action, typically with 5–6 DOF per node, including the in-plane displacements u and v .

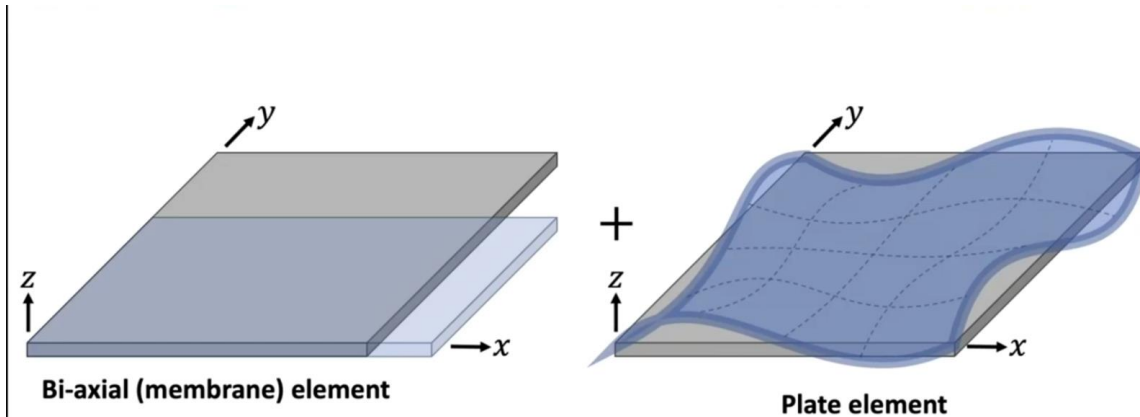


Figure 9 Shell vs Plate Bending Element

However, in our case the load is purely lateral, so there is **no in-plane deformation** (u and v can be validly ignored). Using a shell element would mean adopting a more complex element than the problem requires. Since the Kirchhoff plate assumptions hold and there is no in-plane load, the simplest valid choice is the **plate bending element**, retaining only the bending degrees of freedom (w , θ_x , θ_y).

III. PrePro

Boundary Condition

The boundary conditions applied to the model are shown in Figure Z. The plate is restrained at its mounting holes (green markers), which represent the bolted connections that fix it to the robot frame. At each of these support locations all degrees of freedom are constrained, i.e. the transverse displacement and both bending rotations are set to zero ($w = \theta_x = \theta_y = 0$).

The operational loading is applied as transverse forces (blue arrows) distributed over the plate, representing the weight of the components carried by it. These loads act perpendicular to the plate surface, which is consistent with the purely lateral (bending) load case identified in the problem-definition stage — there is no significant in-plane loading. This combination of fixed mounting points and transverse loads defines the load case used for the deflection and stress analysis.

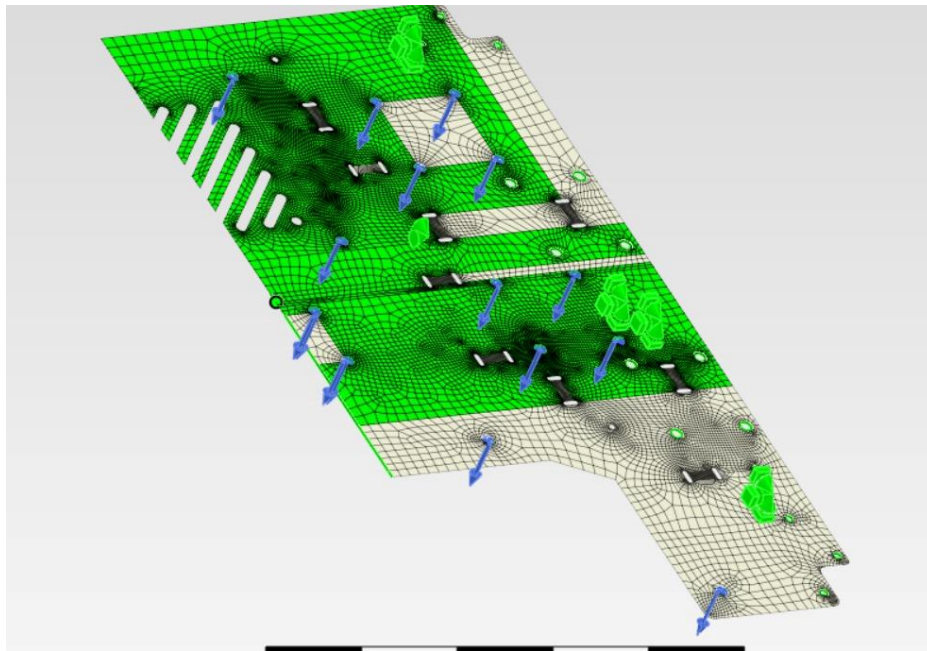


Figure 10 Boundary Conditions

Fixation

For the 6 Holes for each wheel (4 for bearing case that holds the trailing arm + 2 for the Lsection Holds the suspension)

Loading

Driver = 50 gm

Mobile =200 gm

Selfie Stick + 3D Printing Fixation=100 gm

PCB=150 gm

Battery Pack = 500 gm

Drawing on Free Cad

The geometry was discretised with a quadrilateral-dominant mesh (Figure 12), consistent with the four-node (Q4) plate-bending element used in the solver. The mesh is locally refined around every hole and slot, where stress concentrations are expected, and coarsens away from these features to keep the model size manageable. The left edge, highlighted in green, is taken as the fixed (clamped) boundary, and the applied load is introduced at the location marked by the green arrow.

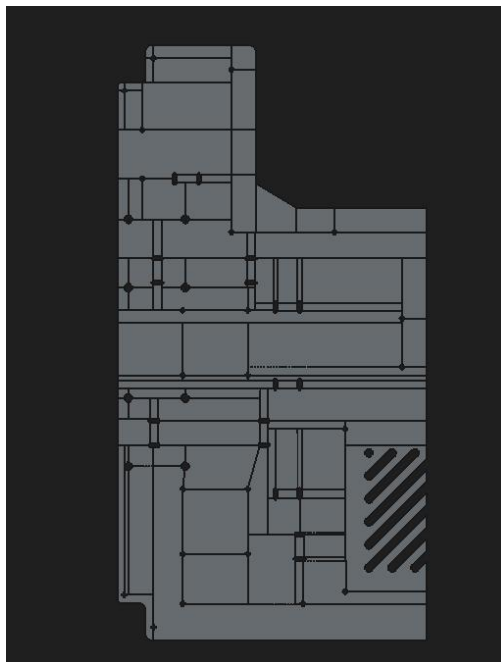


Figure 11 Free CAD

Meshing on Gmsh

The geometry was discretised with a quadrilateral-dominant mesh (Figure 12), consistent with the four-node (Q4) plate-bending element used in the solver. The mesh is locally refined around every hole and slot, where stress concentrations are expected, and coarsens away from these features to keep the model size manageable. The left edge, highlighted in green, is taken as the fixed (clamped) boundary, and the applied load is introduced at the location marked by the green arrow.

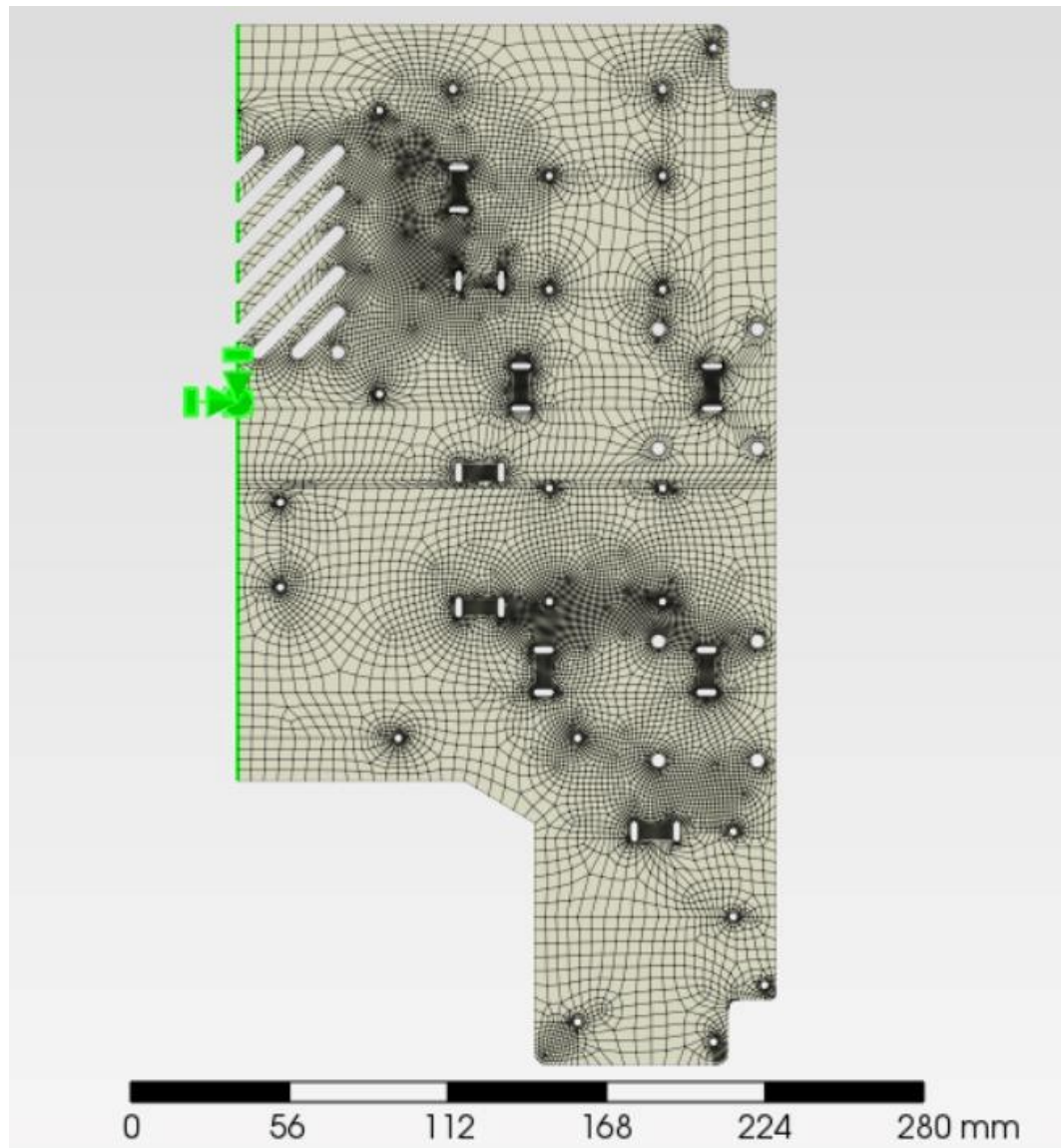


Figure 12 Gmsh

PostProcessing on PrePro

Figure 12 shows the deflection field of the plate under the applied transverse loading with the left edge clamped. The displacement is greatest in the lower-central region of the plate, away from the supports and the stiffening effect of the surrounding holes, and falls to zero at the clamped edge as expected. The maximum deflection is **0.0599 mm**, occurring at node 572.

This deflection is very small compared with both the plate thickness (1.5 mm) and its in-plane dimensions (≈ 280 mm): the ratio of maximum deflection to thickness is only about 0.04. This confirms that the small-deflection assumption underlying the Kirchhoff plate theory used in this analysis is valid, and that the plate is comfortably stiff under the design load.

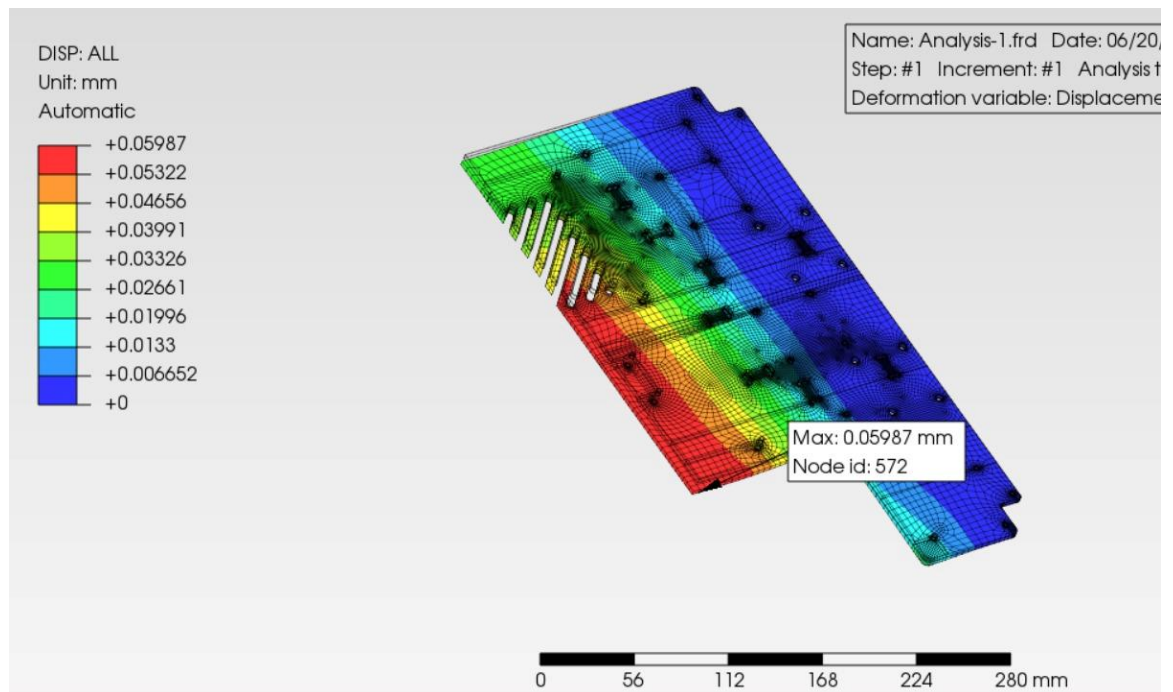


Figure 13 prepro results

IV. MATLAB

Numerical method

Numerical method (weighted sum by gaussian) is used instead of integration due to high complexity of the integration will lead to high computational power.

$$\int_{-1}^1 f(x) dx \approx \sum_{i=1}^n w_i f(x_i)$$

Figure 14 Gaussian

Some low-order quadrature rules are tabulated below for integrating over $[-1, 1]$.

Number of points, n	Points, x_i		Weights, w_i	
1	0		2	
2	$\pm \frac{1}{\sqrt{3}}$	$\pm 0.57735\dots$	1	
3	0		$\frac{8}{9}$	0.888889...
	$\pm \sqrt{\frac{3}{5}}$	$\pm 0.774597\dots$	$\frac{5}{9}$	0.555556...
4	$\pm \sqrt{\frac{3}{7} - \frac{2}{7}\sqrt{\frac{6}{5}}}$	$\pm 0.339981\dots$	$\frac{18 + \sqrt{30}}{36}$	0.652145...
	$\pm \sqrt{\frac{3}{7} + \frac{2}{7}\sqrt{\frac{6}{5}}}$	$\pm 0.861136\dots$	$\frac{18 - \sqrt{30}}{36}$	0.347855...
5	0		$\frac{128}{225}$	0.568889...
	$\pm \frac{1}{3}\sqrt{5 - 2\sqrt{\frac{10}{7}}}$	$\pm 0.538469\dots$	$\frac{322 + 13\sqrt{70}}{900}$	0.478629...
	$\pm \frac{1}{3}\sqrt{5 + 2\sqrt{\frac{10}{7}}}$	$\pm 0.90618\dots$	$\frac{322 - 13\sqrt{70}}{900}$	0.236927...

Figure 15 parameters

MATLAB Results

Nodes: 19030

Elemnts:18052

Max Deflection: 0.0112mm

Max Von Mises stress= 1.801 mpa

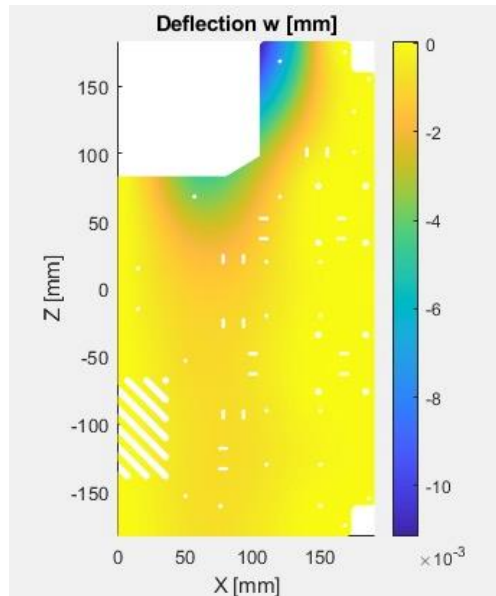


Figure 16 Deflection Result MATLAB

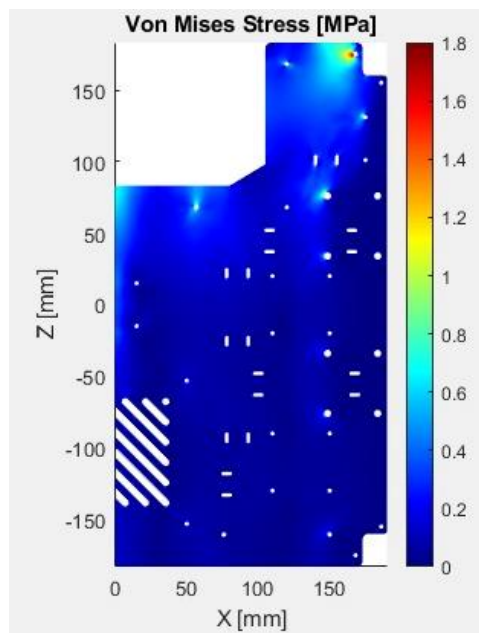


Figure 17 VON MISES Results MATLAB

Appendix A — MATLAB Implementation of the Plate Bending Element

The finite element analysis of the chassis plate was implemented in MATLAB following the plate-bending formulation of Chapter 12 of D. L. Logan, *A First Course in the Finite Element Method* (5th ed.). The script imports a mesh of the plate generated in Gmsh, assembles the global stiffness matrix from four-node rectangular plate-bending elements (12 degrees of freedom each), applies the transverse point loads and the fixed supports at the mounting points, solves for the nodal deflections and rotations, recovers the bending stresses, and finally checks the resulting safety factor against the yield strength of the material (aluminium 6061-T6).

A.1 Overview of the Solution Procedure

At a high level the code follows the standard displacement-based finite element pipeline:

1. **Define material and geometry:** elastic constants, thickness, and the flexural rigidity that forms the constitutive matrix $[D]$.
2. **Import the mesh:** read the nodal coordinates and the quadrilateral connectivity, then assign three degrees of freedom to every node.
3. **Assemble the global stiffness matrix:** loop over every element, build its 12×12 stiffness matrix by Gauss integration, and scatter it into a sparse global matrix.
4. **Apply loads and boundary conditions:** place the transverse loads on the out-of-plane degree of freedom and clamp the mounting nodes.
5. **Solve** $K U = F$ for the free degrees of freedom and extract the deflection field.
6. **Recover stresses and check safety:** compute the bending moments, convert them to surface stresses, evaluate the von Mises stress, and compare against the allowable value.
7. **Post-process:** plot the deflection and stress contours and the safety-factor summary.

A.2 Explanation of the Main Code Blocks

Material properties and flexural rigidity

The first block defines the material of the plate (aluminium 6061-T6) and the design targets. The flexural rigidity is computed as $D = E \cdot t^3 / [12(1-\nu^2)]$ and is used to build the constitutive matrix D_mat , which relates the bending and twisting moments to the curvatures. The factor $(1-\nu)/2$ in the last entry is the shear term of the plane-stress constitutive law. This matrix is the direct numerical form of the moment–curvature relation introduced in the theory.

Mesh import and degrees of freedom

The mesh produced in Gmsh is read in through the generated `first_mesh_matlab` file. Because the plate physically lies in the X–Z plane (the Y-coordinate equals the 1.5 mm thickness), only the first and third coordinate columns are kept as the in-plane coordinates, and the values are converted from millimetres to metres. The connectivity stores four-node quadrilateral (Q4) elements. Each node is given three degrees of freedom — the transverse displacement w and the two bending rotations θ_x and θ_y — so the total number of unknowns is $N_{dof} = 3 \cdot N_{nodes}$ and each element contributes 12 degrees of freedom.

Loads and boundary conditions

Loads and supports are specified by node number rather than by coordinate, which avoids any origin mismatch between the CAD model and the mesh. Each transverse point load is applied to the out-of-plane degree of freedom of its node — the w -DOF, located at global index $3 \times \text{node} - 2$ — with a negative sign because the load acts downward. At every fixed (mounting) node all three degrees of freedom are restrained, i.e. $w = \theta_x = \theta_y = 0$. The remaining degrees of freedom are collected as the free set to be solved.

Gauss quadrature

A 3×3 (nine-point) Gauss rule is used to integrate each element stiffness matrix. Three points per direction are required because the curvature–displacement matrix $[B]$ contains quadratic terms in x and y ; the integrand $B^T D B$ is therefore of high polynomial order, and a lower-order rule would under-integrate it.

Element stiffness matrix assembly

This is the core of the program. For each element the code first builds the 12×12 matrix $[C]$ by evaluating the displacement polynomial and its derivatives (the rows returned by P_matrix) at the four corner nodes; this expresses the nodal degrees of freedom in terms of the 12 polynomial coefficients, $\{d\} = [C]\{\alpha\}$. Inverting it gives $[C]^{-1}$. At every Gauss point the curvature–displacement matrix is formed as $[B] = [Q][C]^{-1}$, and the element stiffness is accumulated as $K_e = \sum B^T D B w_g (b \cdot c)$, where b and c are the element half-dimensions and $b \cdot c$ is the Jacobian. Because the plate mesh is large, the global matrix is stored in sparse triplet form (row, column, value lists) rather than as a dense array, which would not fit in memory.

Solving the system and extracting deflections

After assembly, the system $K U = F$ is solved only over the free degrees of freedom using MATLAB's backslash operator, $U(\text{FreeDof}) = K(\text{FreeDof}, \text{FreeDof}) \setminus F(\text{FreeDof})$. The transverse deflection at every node is then extracted as every third entry of the solution vector (starting at index 1), and the maximum deflection is reported in millimetres.

Stress recovery and safety factor

Stresses are recovered element by element. The bending moments at the element centroid are obtained from $\{M\} = [D][B]\{d\}$, and the corresponding surface stresses follow from $\sigma = 6M / t^2$, which is the maximum fibre stress for a plate of thickness t . The von Mises stress is computed from the three stress components and averaged at the shared nodes for a smooth field. The safety factor is finally $SF = \sigma_y / \sigma_{\max}$, and the script prints a PASS/FAIL verdict against the required minimum.

Post-processing

The final block produces three figures: a contour of the transverse deflection, a contour of the von Mises stress, and a bar chart comparing the maximum stress with the yield and allowable stresses for a quick visual safety check.

The shape-function core: **P_matrix** and **Q_matrix**

The two local functions at the end of the script are where the theory becomes code. **P_matrix** returns three rows: the first row is the full 12-term displacement polynomial for $w(x,y)$ — $[1, x, y, x^2, xy, y^2, x^3, x^2y, xy^2, y^3, x^3y, xy^3]$ — exactly the terms taken from Pascal's triangle, including the two quartic terms x^3y and xy^3 that were selected to preserve displacement continuity between elements. The second and third rows are the rotations $\theta_x = +\partial w/\partial y$ and $\theta_y = -\partial w/\partial x$, i.e. the first derivatives of that same polynomial. **Q_matrix** returns the curvatures $\kappa_x = -\partial^2 w/\partial x^2$, $\kappa_y = -\partial^2 w/\partial y^2$ and $\kappa_{xy} = -2\partial^2 w/\partial x\partial y$ — the second derivatives of the displacement polynomial. These are the same curvatures defined in the theory, and combining them with $[C]^{-1}$ gives the strain field that drives the whole stiffness calculation.

A.3 Mapping Between Code and Theory

The table below links the main objects in the code to the quantities developed in the theory section.

Code object	What it represents	Theory section
E, nu, t	Elastic constants and thickness of the Al 6061-T6 plate	Problem definition
D_mat	Constitutive (flexural) matrix [D] linking moments to curvatures	Constitutive relation
Ndof= 3*Nnodes	3 DOF per node (w, theta_x, theta_y), 12 DOF per element	Element & DOF
P_matrix row 1	The 12-term displacement polynomial $w(x,y)$	Displacement field (Pascal)
x^3y , xy^3	The two quartic terms chosen for inter-element continuity	Polynomial selection
Q_matrix	Curvature-displacement operator [Q]: $\kappa_x, \kappa_y, \kappa_{xy}$	Curvatures
$B=Q_matrix*C_inv$	Curvature-displacement matrix $[B] = [Q][C]^{-1}$	Formulation
$Ke=B'*D_mat*B*...$	Element stiffness, integrated by Gauss quadrature	Stiffness formulation
$6*M_e/t^2$	Recovery of surface bending stress from moments	Stress recovery

A.4 Complete MATLAB Listing

The complete source file (plate_bending_Logan_Ch12.m) is reproduced below.

```
%Finite Element Analysis of Mobile Robot Chassis Plate (Plate Bending)

%Reference: D. L. Logan, "A First Course in the Finite Element Method"
%           5th Edition, Chapter 12 : Plate Bending Element

clear; clc; close all;

%% Parameters & Loads

E      = 69e9;      %Youngs Modulus [Pa]   (Al 6061-T6)
nu     = 0.33;      %Poisson's ratio
sigma_y = 276e6;    %Yield strength [Pa]
SF_min = 2.0;       %Required safety factor
t      = 1.5e-3;    %Plate thickness [m]   (from mesh Y-coordinate)

%Bending Rigidity  $D = E*t^3/[12(1-\nu^2)]$  --> Logan Eq.(12.2.18)
D_flex = E*t^3/(12*(1-nu^2));
D_mat = D_flex*[1, nu, 0; nu, 1, 0; 0, 0, (1-nu)/2];

%% Mesh Import (Gmsh / Prepro)

mesh_file = 'first_mesh_matlab'; %put this file in the same folder
run(mesh_file);

%Plate lies in X-Z plane (Y = 1.5mm = thickness) -> use cols 1 and 3
nodes = [msh.POS(:,1), msh.POS(:,3)] * 1e-3; %mm -> m [Nnodes x 2]
connectivity = msh.QUADS(:,1:4); %Q4 elements [Nelem x 4]

Nnodes = size(nodes,1);
Nelem = size(connectivity,1);
Ndof = 3*Nnodes; %3 DOF per node: w, thetax, thetay (Logan Step1)

%--- External point loads (manual, like Fdrag/Fcent in the truss code) ---
```

```

%Each row: [node_number, Fz (N)] -- Fz downward, EDIT to your loads

%Node numbers are read directly from the mesh (avoids origin/coordinate
%mismatch issues between CAD and mesh files)

point_loads = [
    140,    5.0;
    275,    8.0;
    9201,   3.0;
];

n_loads = size(point_loads,1);

%--- Fixed boundary condition points (manual, like FixedDof in truss code) ---

%Each row: [node_number] -- node to be fixed (bolt hole / mount point)

%At each node: w = thetax = thetay = 0 (EDIT to your actual mount nodes)

fixed_points = [
    1;
    50;
    12500;
    155684;
];

n_fixed = size(fixed_points,1);

%% Gauss Quadrature 3x3 (9 points) -- needed since B has x^2,y^2 terms

gp3 = [-sqrt(3/5), 0, sqrt(3/5)];
gw3 = [5/9, 8/9, 5/9];

GP = zeros(9,2); GW = zeros(9,1); idx=0;
for ii=1:3
    for jj=1:3
        idx=idx+1;
        GP(idx,:)=[gp3(ii), gp3(jj)];
        GW(idx)=gw3(ii)*gw3(jj);
    end
end
end

```



```

%% Global Stiffness Matrix Assembly

%Sparse triplet storage (Nnodes is large -> dense K is impossible in RAM)

K_rows = []; K_cols = []; K_vals = [];

F = zeros(Ndof,1);

for e = 1:Nelem

    n1 = connectivity(e,1);
    n2 = connectivity(e,2);
    n3 = connectivity(e,3);
    n4 = connectivity(e,4);

    xe = nodes([n1 n2 n3 n4], 1);
    ye = nodes([n1 n2 n3 n4], 2);

    %Half-dimensions of the (rectangular) Q4 element -- Logan uses b,c
    b = (max(xe)-min(xe))/2;
    c = (max(ye)-min(ye))/2;
    x0 = min(xe); y0 = min(ye);

    if b < 1e-12 || c < 1e-12
        continue
    end

    %--- [C] matrix (12x12) -- Logan Eq.(12.2.8): {d} = [C]{alpha} ---
    C_mat = zeros(12,12);

    for nd = 1:4

        rows = (nd-1)*3 + (1:3);

        C_mat(rows,:) = P_matrix(xe(nd), ye(nd));

    end

    if abs(det(C_mat)) < 1e-30

```

```

        continue

    end

    C_inv = inv(C_mat);    %[C]^-1 -- Logan Eq.(12.2.10)

    %--- Element stiffness Ke -- Logan Eq.(12.2.19) ---

    Ke = zeros(12,12);

    for g = 1:9

        x_g = x0 + b*(1+GP(g,1));

        y_g = y0 + c*(1+GP(g,2));

        B = Q_matrix(x_g,y_g) * C_inv;    %[B]=[Q][C]^-1 -- Eq.(12.2.16)

        Ke = Ke + B'*D_mat*B*GW(g)*b*c;    %Jacobian = b*c

    end

    dof = [3*n1-2 3*n1-1 3*n1, 3*n2-2 3*n2-1 3*n2, ...
           3*n3-2 3*n3-1 3*n3, 3*n4-2 3*n4-1 3*n4];

    for i = 1:12

        for j = 1:12

            K_rows(end+1) = dof(i);

            K_cols(end+1) = dof(j);

            K_vals(end+1) = Ke(i,j);

        end

    end

end

K = sparse(K_rows, K_cols, K_vals, Ndof, Ndof);

%% Force Vector

%Manual applying of point loads (like the Fdrag/Fcent loop in truss code)

%Each load applied directly on the w-DOF (= 3*node-2) of the given node

x_all = nodes(:,1); y_all = nodes(:,2);

```

```

for i = 1:n_loads
    node_i = point_loads(i,1);
    F1      = point_loads(i,2);

    F(3*node_i-2) = F(3*node_i-2) - F1;    %downward load on w-DOF
end

%% Boundary Condition
%Manual Set For Fixed & Free DOF -- at user-defined node numbers
%At each fixed node: w = thetax = thetay = 0

FixedDof = [];
for c = 1:n_fixed
    node_c = fixed_points(c);
    FixedDof = [FixedDof, 3*node_c-2, 3*node_c-1, 3*node_c];
end

FreeDof = setdiff(1:Ndof, FixedDof);

%% EQ --> KU=F
%Nodal Displacement
U = zeros(Ndof,1);
U(FreeDof) = K(FreeDof,FreeDof) \ F(FreeDof);

w_node = U(1:3:end);
w_max  = max(abs(w_node))*1000;    %mm

%% Stress (Logan Eq.12.1.8 + Eq.12.2.17)
vm_node = zeros(Nnodes,1);
node_count = zeros(Nnodes,1);

```

```

for e = 1:Nelem

    n1 = connectivity(e,1); n2 = connectivity(e,2);

    n3 = connectivity(e,3); n4 = connectivity(e,4);

    xe = nodes([n1 n2 n3 n4],1);
    ye = nodes([n1 n2 n3 n4],2);

    b = (max(xe)-min(xe))/2; c = (max(ye)-min(ye))/2;
    x0 = min(xe); y0 = min(ye);

    if b<1e-12 || c<1e-12; continue; end

    C_mat = zeros(12,12);

    for nd=1:4

        rows=(nd-1)*3+(1:3);

        C_mat(rows,:) = P_matrix(xe(nd),ye(nd));

    end

    if abs(det(C_mat))<1e-30; continue; end

    C_inv = inv(C_mat);

    dof = [3*n1-2 3*n1-1 3*n1, 3*n2-2 3*n2-1 3*n2, ...
           3*n3-2 3*n3-1 3*n3, 3*n4-2 3*n4-1 3*n4];

    d_e = U(dof);

    xc = x0+b; yc = y0+c; %evaluate at element centroid

    B = Q_matrix(xc,yc)*C_inv;

    M_e = D_mat*B*d_e; %{M}=[D][B]{d} -- Logan Eq.(12.2.17)

    sx = 6*M_e(1)/t^2; %Logan Eq.(12.1.8)
    sy = 6*M_e(2)/t^2;
    txy = 6*M_e(3)/t^2;

```

```

vm = sqrt(sx^2 - sx*sy + sy^2 + 3*txy^2);

for nd = [n1 n2 n3 n4]
    vm_node(nd) = vm_node(nd) + vm;
    node_count(nd) = node_count(nd) + 1;
end
end
vm_node = vm_node ./ max(node_count,1);

[Maxstress, Max_stress_nodeNo] = max(vm_node);
SF = sigma_y / Maxstress;

%% Display Data
fprintf('Nodes      : %d\n', Nnodes);
fprintf('Elements : %d\n', Nelem);
fprintf('Max deflection w      = %.4f mm\n', w_max);
fprintf('Max Von Mises stress = %.3f MPa\n', Maxstress/1e6);
fprintf('Safety Factor SF      = %.2f (required >= %.1f)\n', SF, SF_min);
if SF >= SF_min
    fprintf('STATUS: PASS\n');
else
    fprintf('STATUS: FAIL -- increase thickness\n');
end

%% Plotting
figure; hold on;
patch('Faces',connectivity,'Vertices',nodes, ...
    'FaceVertexCData', w_node*1000, ...
    'FaceColor','interp','EdgeColor','none');
colormap(parula); colorbar;

```

```

axis equal tight; xlabel('X [m]'); ylabel('Y [m]');
title('Deflection w [mm]');

figure; hold on;
patch('Faces',connectivity,'Vertices',nodes, ...
      'FaceVertexCData', vm_node/1e6, ...
      'FaceColor','interp','EdgeColor','none');
colormap(jet); colorbar;
axis equal tight; xlabel('X [m]'); ylabel('Y [m]');
title('Von Mises Stress [MPa]');

figure; hold on;
vals = [Maxstress/1e6, sigma_y/1e6, sigma_y/(SF_min*1e6)];
labels = {'sigma_{VM}','Yield','Allowable'};
bar(vals,0.5);
set(gca,'XTickLabel',labels);
ylabel('Stress [MPa]'); grid on;
title('Safety Factor Check');

%% Local Functions -- Logan Ch.12 derivation
function P = P_matrix(x,y)
%Polynomial matrix -- Logan Eq.(12.2.6)
%Row1: w , Row2: thetax=+dw/dy , Row3: thetay=-dw/dx
P = zeros(3,12);
P(1,:) = [1, x, y, x^2, x*y, y^2, x^3, x^2*y, x*y^2, y^3, x^3*y, x*y^3];
P(2,:) = [0, 0, 1, 0, x, 2*y, 0, x^2, 2*x*y, 3*y^2, x^3, 3*x*y^2];
P(3,:) = [0,-1, 0,-2*x,-y, 0,-3*x^2,-2*x*y,-y^2, 0,-3*x^2*y,-y^3];
end

function Q = Q_matrix(x,y)
%Curvature matrix -- Logan Eq.(12.2.13)

```

```
%kx=-d2w/dx2 , ky=-d2w/dy2 , kxy=-2*d2w/dxdy

Q = zeros(3,12);

Q(1,:) = [0,0,0,-2,0,0,-6*x,-2*y,0,0,-6*x*y,0];
Q(2,:) = [0,0,0,0,0,-2,0,0,-2*x,-6*y,0,-6*x*y];
Q(3,:) = [0,0,0,0,-2,0,0,-4*x,-4*y,0,-6*x^2,-6*y^2];

end
```